

SOCIALSPHEAR

Technical Architecture Specification

Version 1.0 | June 2026 | Confidential

1. System Overview

This document defines the complete technical architecture for SocialSphear — a next-generation social media management and scheduling SaaS platform. It covers infrastructure topology, data models, service design, scheduling engine internals, AI integration, security architecture, and operational concerns.

PRINCIPLE	The entire platform is TypeScript end-to-end. One language from database schema (Prisma) through API layer (tRPC + NestJS) to UI components (Next.js). Shared types live in a /packages/types package in the Turborepo monorepo and are imported by all services.
-----------	---

1.1 High-Level Architecture

Layer	Technology	Responsibility
Client (Web)	Next.js 15 App Router	UI rendering, client state, real-time updates via WebSocket
API Gateway	NestJS + tRPC router	Auth, input validation, rate limiting, route dispatch
Scheduling Engine	BullMQ + Temporal	Reliable post delivery, retries, workflow orchestration
Data Store	PostgreSQL (Supabase)	All relational business data — users, posts, accounts, workspaces
Cache	Redis (Upstash)	Sessions, API response cache, rate limit counters, job state
Analytics Store	ClickHouse Cloud	Post performance metrics, aggregated engagement data
Media Pipeline	Cloudflare R2 + Cloudinary	Raw upload storage and per-platform image transformation
AI Services	Anthropic Claude + fal.ai	Caption generation, brand voice, image generation
Search	Typesense	Content library search, hashtag autocomplete
Auth	Clerk	User identity, OAuth management, team/org management
CDN + Edge	Cloudflare	Static assets, DDoS protection, WAF, DNS
Email	Resend	Transactional and notification emails

1.2 Monorepo Structure

```
socialsphear/
├── apps/
│   ├── web/           # Next.js 15 frontend
│   └── api/           # NestJS backend
├── packages/
└── types/             # Shared TypeScript types and Zod schemas
```

```
| | | db/           # Prisma schema, client, migrations
| | | ui/           # Shared Shadcn/UI component library
| | | config/       # ESLint, TypeScript, Tailwind base configs
| | | ai/           # Shared AI SDK wrappers (Claude, fal.ai)
| | turbo.json
| | package.json
| | pnpm-workspace.yaml
```

2. Frontend Architecture

2.1 Next.js App Router Structure

```
apps/web/src/
├── app/
│   ├── (auth)/
│   │   ├── sign-in/      # Clerk sign-in page
│   │   └── sign-up/      # Clerk sign-up page
│   ├── (dashboard)/
│   │   ├── layout.tsx    # Dashboard shell with sidebar and header
│   │   ├── page.tsx      # Dashboard home – overview stats
│   │   ├── calendar/     # Content calendar views
│   │   ├── composer/     # Post composer
│   │   ├── analytics/    # Analytics dashboard
│   │   ├── inbox/        # Social inbox
│   │   ├── ai-studio/    # AI content creation tools
│   │   └── settings/     # Workspace and account settings
│   └── api/              # Next.js API routes (webhooks, uploads)
├── components/
│   ├── calendar/         # DnD calendar components
│   ├── composer/         # Rich text editor, media uploader, preview
│   ├── analytics/        # Chart components (Recharts)
│   └── ui/               # Shadcn/UI components
├── hooks/                # Custom React hooks
├── lib/
│   ├── trpc.ts           # tRPC client config
│   └── socket.ts         # Socket.io client setup
└── stores/              # Zustand state stores
```

2.2 State Architecture

State Type	Tool	Examples
Server state	TanStack Query v5	Post list, analytics data, social accounts, workspace members
Global UI state	Zustand	Active workspace, selected platform filters, composer draft, sidebar state
Form state	React Hook Form + Zod	Composer form, settings forms, invite forms
Real-time state	Socket.io + Zustand	Post publish status, team member activity, notification count
URL state	Next.js searchParams	Calendar date range, analytics filters, active tab

2.3 Key Frontend Patterns

- Optimistic updates — schedule a post and it appears on the calendar immediately before server confirmation
- Streaming AI — captions stream token-by-token using Vercel AI SDK useCompletion hook

- Virtual rendering — calendar with 500+ posts uses TanStack Virtual for DOM efficiency
- Image lazy loading — all media uses Next.js Image component with blur placeholder
- Error boundaries — each dashboard section has an isolated error boundary to prevent full-page crashes
- Skeleton loading — every data-fetching component renders skeleton UI while loading

3. Backend Architecture

3.1 NestJS Module Structure

```

apps/api/src/
├── app.module.ts
├── modules/
│   ├── auth/                # Clerk JWT verification guard
│   ├── workspace/           # Workspace CRUD, member management
│   ├── social-accounts/     # OAuth connect/disconnect, token refresh
│   ├── posts/               # Post creation, editing, deletion
│   ├── scheduler/           # BullMQ producers, job management
│   ├── publisher/           # BullMQ consumers – actual API publishing
│   ├── analytics/           # ClickHouse ingestion and query service
│   ├── ai/                  # Caption gen, image gen, brand voice
│   ├── media/               # R2 upload, Cloudinary transforms
│   ├── search/              # Typesense indexing and query
│   ├── notifications/       # Socket.io events, email triggers
│   ├── billing/             # Stripe webhooks, plan management
│   └── webhooks/            # Inbound webhooks (social platforms)
├── trpc/
│   ├── router.ts            # Root tRPC router
│   └── routers/             # Per-domain tRPC routers

```

3.2 tRPC API Design

tRPC Router	Key Procedures	Auth Required
workspace	create, update, delete, getMembers, invite, updateMemberRole	Yes
posts	create, update, delete, schedule, getByWorkspace, getCalendar, duplicate	Yes
socialAccounts	connect, disconnect, refresh, list, getHealth	Yes
analytics	getPostMetrics, getAccountMetrics, getTopPosts, exportCSV	Yes
ai	generateCaption, generateImage, trainBrandVoice, suggestHashtags, suggestBestTime	Yes
media	getUploadUrl, confirmUpload, deleteMedia, listMediaLibrary	Yes
billing	getPlans, createCheckout, getSubscription, cancelSubscription	Yes
notifications	list, markRead, markAllRead	Yes

3.3 Auth Flow

All requests to NestJS carry a Clerk session JWT in the Authorization header. The ClerkGuard NestJS guard verifies the JWT, extracts the userId and orgId, and attaches them to the request

context. All downstream service calls use these verified identities — no trust is placed on client-supplied user IDs.

```
// ClerkGuard – verifies JWT and attaches user context
@Inject()
export class ClerkGuard implements CanActivate {
  async canActivate(context: ExecutionContext): Promise<boolean> {
    const request = context.switchToHttp().getRequest();
    const token = extractBearerToken(request);
    const session = await clerkClient.verifyToken(token);
    request.userId = session.sub;
    request.orgId = session.org_id;
    return true;
  }
}
```

4. Scheduling Engine

CRITICAL

The scheduling engine is the most important system in SocialSphear. A post that fails to publish — or publishes late — destroys user trust. Every design decision here prioritises reliability above all else.

4.1 BullMQ Queue Architecture

Queue	Purpose	Concurrency	Retry Strategy
post:schedule	Receives new scheduled post jobs	50	No retry — idempotent entry point
post:publish	Executes API call to social platform	20 per platform	3 retries, exponential backoff (30s, 2m, 10m)
post:notify	Sends success/failure notifications	100	2 retries, 5s delay
analytics:ingest	Fetches post metrics from platform APIs	10	5 retries, 1h delay
token:refresh	Refreshes expiring OAuth tokens	5	3 retries, 1m delay
media:process	Triggers Cloudinary transforms after upload	30	2 retries, 10s delay

4.2 Post Lifecycle State Machine

```

DRAFT
|
v
SCHEDULED --> QUEUED --> PUBLISHING --> PUBLISHED
                                   |
                                   +--> FAILED --> RETRYING
                                   |
                                   +--> CANCELLED      +--> FAILED_PERMANENT
  
```

4.3 Temporal Workflows

Workflow	Trigger	Steps
ScheduledPostWorkflow	User schedules a post	1. Wait until scheduledAt 2. Dispatch to BullMQ 3. Monitor delivery 4. Ingest analytics after 1h 5. Send engagement summary after 24h
ThreadPostWorkflow	User schedules a Twitter thread	1. Post first tweet 2. Wait 2s 3. Post reply tweet 4. Repeat for each thread item 5. Record thread URL
RecurringPostWorkflow	User sets weekly recurring post	1. Schedule current instance 2. Monitor completion 3. Clone post 4. Set next occurrence 5. Repeat indefinitely

Workflow	Trigger	Steps
TokenRefreshWorkflow	OAuth token expires in under 7 days	1. Attempt token refresh 2. If fails, notify user 3. Retry daily 4. Suspend scheduled posts after 3 failures

5. Database Design

5.1 PostgreSQL Schema (Core Tables)

```
-- Users (managed primarily by Clerk, mirrored here for FK references)
CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  clerk_id    TEXT UNIQUE NOT NULL,
  email       TEXT UNIQUE NOT NULL,
  name        TEXT,
  avatar_url  TEXT,
  created_at  TIMESTAMPTZ DEFAULT now()
);

-- Workspaces (one per brand or client)
CREATE TABLE workspaces (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT NOT NULL,
  slug        TEXT UNIQUE NOT NULL,
  logo_url    TEXT,
  owner_id    UUID REFERENCES users(id),
  plan        TEXT NOT NULL DEFAULT 'free',
  created_at  TIMESTAMPTZ DEFAULT now()
);

-- Social Accounts (one row per connected account)
CREATE TABLE social_accounts (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id UUID REFERENCES workspaces(id) ON DELETE CASCADE,
  platform    TEXT NOT NULL,
  platform_user_id TEXT NOT NULL,
  username    TEXT,
  access_token BYTEA NOT NULL,      -- AES-256 encrypted
  refresh_token BYTEA,              -- AES-256 encrypted
  token_expires_at TIMESTAMPTZ,
  status      TEXT DEFAULT 'active',
  created_at  TIMESTAMPTZ DEFAULT now(),
  UNIQUE(workspace_id, platform, platform_user_id)
);

-- Posts (the content unit)
CREATE TABLE posts (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id UUID REFERENCES workspaces(id) ON DELETE CASCADE,
  created_by  UUID REFERENCES users(id),
  content     TEXT,
  media_urls  JSONB DEFAULT '[]',
  platform_overrides JSONB DEFAULT '{}',
  scheduled_at TIMESTAMPTZ,
  status      TEXT DEFAULT 'draft',
  approval_status TEXT DEFAULT 'none',
  created_at  TIMESTAMPTZ DEFAULT now(),
  updated_at  TIMESTAMPTZ DEFAULT now()
);

-- Post Jobs (one per platform per post)
CREATE TABLE post_jobs (
```

```

    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    post_id            UUID REFERENCES posts(id) ON DELETE CASCADE,
    social_account_id  UUID REFERENCES social_accounts(id),
    platform           TEXT NOT NULL,
    status             TEXT DEFAULT 'pending',
    platform_post_id   TEXT,
    platform_url        TEXT,
    published_at       TIMESTAMPTZ,
    error_message       TEXT,
    retry_count         INTEGER DEFAULT 0,
    created_at         TIMESTAMPTZ DEFAULT now()
);

-- Team Members (workspace access)
CREATE TABLE team_members (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id       UUID REFERENCES workspaces(id) ON DELETE CASCADE,
    user_id            UUID REFERENCES users(id) ON DELETE CASCADE,
    role               TEXT NOT NULL DEFAULT 'editor',
    invite_status       TEXT DEFAULT 'pending',
    joined_at          TIMESTAMPTZ,
    UNIQUE(workspace_id, user_id)
);

```

5.2 ClickHouse Analytics Schema

```

-- Post metrics – one row per metric snapshot
CREATE TABLE post_metrics (
    post_job_id        UUID,
    workspace_id        UUID,
    platform            LowCardinality(String),
    recorded_at         DateTime,
    likes               UInt32 DEFAULT 0,
    comments            UInt32 DEFAULT 0,
    shares              UInt32 DEFAULT 0,
    saves               UInt32 DEFAULT 0,
    reach               UInt32 DEFAULT 0,
    impressions         UInt32 DEFAULT 0,
    engagement_rate     Float32
) ENGINE = MergeTree()
  PARTITION BY toYYYYMM(recorded_at)
  ORDER BY (workspace_id, platform, recorded_at);

-- Account growth – one row per daily snapshot
CREATE TABLE account_snapshots (
    social_account_id  UUID,
    workspace_id        UUID,
    platform            LowCardinality(String),
    snapshot_date       Date,
    follower_count      UInt32,
    following_count     UInt32,
    post_count          UInt32
) ENGINE = MergeTree()
  PARTITION BY toYYYYMM(snapshot_date)
  ORDER BY (workspace_id, platform, snapshot_date);

```

6. AI Architecture

6.1 Caption Generation Pipeline

```
USER REQUEST
  |
  v
[Input Validation] -- platform, tone, topic, brand voice profile
  |
  v
[Context Assembly] -- fetch brand voice examples from DB
  |
  v
[Prompt Construction] -- system prompt + user examples + request
  |
  v
[Anthropic Claude API] -- claude-sonnet-4-6, streaming
  |
  v
[Post-Processing] -- extract hashtags, trim to platform limit
  |
  v
[Stream to Client] -- Vercel AI SDK useCompletion hook
```

6.2 Brand Voice Training

Step	Process	Storage
1. User uploads examples	5–20 sample posts submitted via AI Studio	PostgreSQL — brand_voice_examples table
2. Embedding generation	Claude embedding API generates vector per example	PostgreSQL pgvector extension — 1536-dim vectors
3. Similarity retrieval	On caption request, cosine similarity search for top-3 examples	pgvector IVFFlat index for fast ANN search
4. Few-shot injection	Top-3 examples injected into system prompt as style references	In-memory per request
5. Output validation	Verify output length fits platform limit, re-generate if not	In-memory per request

6.3 AI Credit Costs

Action	Credit Cost	Notes
Generate caption (1 platform)	1 credit	Cached — same input within 1h returns cached result
Generate caption (all platforms)	3 credits	Platform-specific variants
Generate hashtag suggestions	1 credit	Cached per topic per 24h
Generate AI image (512x512)	10 credits	fal.ai Flux model

Action	Credit Cost	Notes
Generate AI image (1024x1024)	20 credits	fal.ai Flux Pro model
Analyse best posting time	2 credits	Uses 90-day analytics history
Generate monthly content plan	15 credits	LangChain agent — multiple AI calls

7. Security Architecture

7.1 OAuth Token Security

- All access tokens and refresh tokens encrypted with AES-256-GCM before database storage
- Encryption key stored in Railway secrets environment variable — never in code or database
- Key rotation: quarterly automated rotation with dual-key decryption window
- Tokens never logged, never returned to frontend, never included in error messages
- Token refresh triggered automatically 24h before expiry via Temporal workflow

7.2 API Security

Threat	Mitigation
Unauthenticated access	ClerkGuard on all NestJS routes — JWT verification on every request
Cross-workspace data leak	Workspace ID always validated against authenticated user's memberships
SQL injection	Prisma parameterised queries — no raw SQL with user input
XSS	React JSX escaping plus Helmet.js Content-Security-Policy header
CSRF	SameSite=Strict cookies plus CSRF token on state-mutating requests
DDoS	Cloudflare WAF plus rate limiting at edge before requests reach NestJS
Brute force	Clerk handles auth — rate-limited by Clerk infrastructure
Secrets exposure	No secrets in client bundle — all AI calls proxied through NestJS

7.3 Rate Limiting

Scope	Limit	Window	Storage
Per-user API calls	500 requests	1 minute	Redis (Upstash)
AI caption generation	10 requests	1 minute	Redis
AI image generation	3 requests	1 minute	Redis
Social account OAuth connect	5 attempts	1 hour	Redis
Post scheduling	100 posts	1 hour	Redis
Media upload	50 files	1 hour	Redis

8. Infrastructure and DevOps

8.1 Environments

Environment	Frontend	Backend	Database	Purpose
Development	localhost:3000	localhost:3001	Local Docker PostgreSQL	Local developer iteration
Preview	Vercel preview URL	Railway PR environment	Supabase branch DB	PR review and QA
Staging	staging.socialsphear.com	api-staging.socialsphear.com	Supabase staging project	Pre-production validation
Production	socialsphear.com	api.socialsphear.com	Supabase production project	Live traffic

8.2 CI/CD Pipeline

```
ON PUSH TO ANY BRANCH:
  1. pnpm lint (ESLint + TypeScript strict check)
  2. pnpm test (Vitest unit tests – all packages)
  3. pnpm build (Turborepo cached build)

ON PULL REQUEST:
  4. Playwright E2E tests (against staging)
  5. Vercel preview deployment (frontend)
  6. Railway preview deployment (backend)
  7. Post preview URLs to PR as GitHub comment

ON MERGE TO MAIN:
  8. Vercel production deployment (automatic)
  9. Railway production deployment (automatic)
  10. Prisma migrate deploy (run migrations)
  11. Sentry release notification
  12. PostHog deployment annotation
```

8.3 Observability Stack

Concern	Tool	What It Monitors
Frontend errors	Sentry	JS exceptions, failed API calls, Web Vitals (LCP, FID, CLS)
Backend errors	Sentry	Unhandled exceptions, slow endpoints, BullMQ job failures
User behaviour	PostHog	Feature usage, conversion funnels, session recordings, A/B tests
Infrastructure	Railway metrics	CPU, memory, network, request latency per service
Queue health	BullMQ dashboard	Job counts, failure rates, processing time per queue

Concern	Tool	What It Monitors
Database	Supabase dashboard	Query performance, connection pool, slow query log
Uptime	Better Uptime	HTTP checks on all public endpoints every 60 seconds

9. Performance Targets

Metric	Target	Measurement Method
Page load (LCP)	< 1.5s	Web Vitals via Sentry, tested on 4G connection
API response (p50)	< 100ms	NestJS response time middleware
API response (p99)	< 500ms	NestJS response time middleware
Calendar render (500 posts)	< 200ms	Playwright performance test
AI caption stream first token	< 800ms	Client-side measurement via PostHog
Post scheduling to queue	< 1s	BullMQ job creation timestamp
Post delivery success rate	> 99.5%	post_jobs status tracking in PostgreSQL
Post delivered on time (within 60s)	> 99%	scheduledAt vs publishedAt delta
ClickHouse analytics query (90d)	< 2s	Server-side timing in NestJS
Monthly uptime	> 99.9%	Better Uptime external monitoring

SCALE TRIGGER

Migrate from Railway to AWS EKS when monthly API requests exceed 50 million or when Railway costs exceed \$2,000/month. At that point, the team will have the revenue and engineering capacity to manage Kubernetes.